

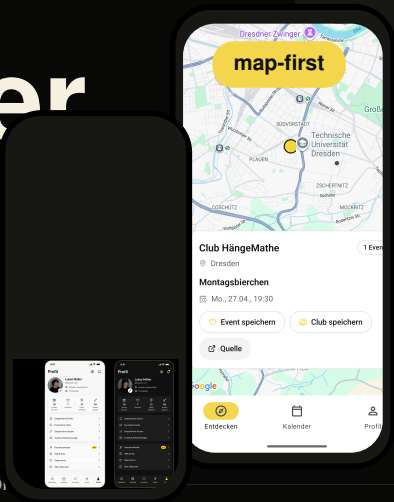
FINAL PROJECT VIDEO

DDiscover

A mobile app for Dresden

HTW Dresden Mobile Networks APL

React Native + Expo Convex Clerk MapLibre



- CONTEXT

Student club events need one mobile entry point.

Problem

Events are spread across club pages, feeds, social posts, and personal knowledge.

The goal is not to build a generic event portal. The goal is to answer the student's practical question: **what is happening nearby, and how do I get there?**

Product answer

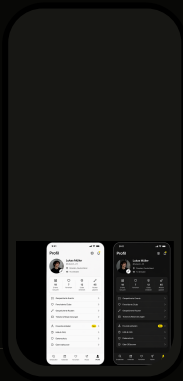
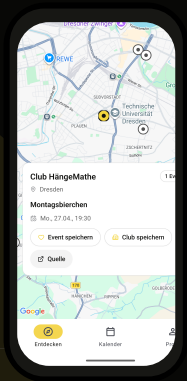
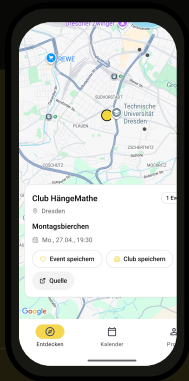
A map-first app that connects events, locations, details, and routes.

MVP focus

Discover, calendar, event details, route planning, profile/auth shell.

• APP OVERVIEW

The app is organized around the real user flow.



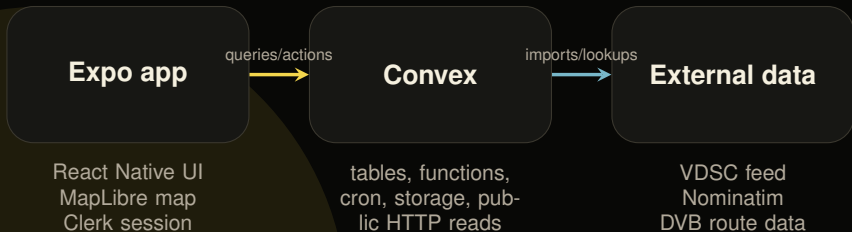
Primary screens

Discover map, calendar, event detail, profile.

- **Discover**
Select a student club on the map
- **Calendar**
Browse upcoming VDSC events
- **Event detail**
Image, source, time, location, route
- **Profile**
Guest-first auth and settings

- **SYSTEM SHAPE**

A small backend owns the hard parts.

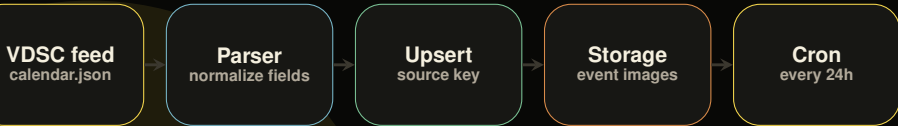


Design choice

Keep mobile screens thin and let Convex handle scheduled imports, normalization, storage, and external service boundaries.

- **IMPORT PIPELINE**

The event data path is idempotent and testable.



The project avoids browser scraping for the core data path. The structured VDSC feed is easier to verify and safer to run in production.

Event images are scraped from detail pages and stored in Convex Storage, so the mobile app reads stable generated HTML.

56+ live VDSC events imported during final testing

- LOCATION RELIABILITY

For maps, one wrong coordinate is visible immediately.

Coordinate priority

- 1 Reviewed Convex**
Manual or checked coordinates win first
- 2 Curated MVP fallback**
Known clubs stay usable offline
- 3 Geocoding cache**
Nominatim only fills gaps

Lesson: automated geocoding is useful enrichment, but reviewed coordinates are the better source of truth for a student-facing map MVP.

- NATIVE RUNTIME

The hardest bug only appeared in APK builds.

Local dev

Metro + dev client worked

Standalone APK

Convex data failed

Fix

HTTP queries + URL normalization

Root causes: unstable Convex function references as effect dependencies, WebSocket path behavior in the APK context, and a trailing slash in the CI Convex URL.

Result: public app data loads through stable Convex HTTP queries; runtime config strips trailing slashes before building API paths.

- TESTING AND CI

Device testing became part of the product work.

17

backend tests

parser, import, geocoding, image

3

core checks

TypeScript, lint, Vitest

2

APK paths

GitHub Actions + GitLab CI

1

phone loop

artifact, install, logs

Expo Go was not enough because the app uses native modules and a dev-client workflow. APK builds make the final behavior testable on a real Android phone and catch missing public configuration before hand-in.

- **DIFFICULTIES**

Most risk came from system boundaries, not screen code.

Android tooling

ADB, emulator disk, stale Metro

state

Data quality

VDSC feed gaps, images, coordi-

Practical takeaway

Mobile projects need early real-device checks, explicit runtime config validation, and written root-cause notes.

Auth handoff

Clerk session vs Convex auth in

ADB

Map interaction

Native markers, drawer gestures,

theme colors

- **FINAL STATE**

The project has a complete end-to-end discovery loop.

Ready to show

- Map-first student club discovery
- Calendar and event detail pages
- Event images stored through Convex
- DVB route planning on event details
- Guest-first profile and auth shell
- Manual APK builds for phone testing

Closing line: DDiscover is no longer just a prototype. It connects real imported events, a mobile discovery UI, backend automation, route planning, and a reproducible submission process.

What remains

Fully re-enable cloud-synced favorites, add richer filters, verify GitLab hand-in settings, and run final APK smoke test.